2º DAW - Optativa “J2EE y Spring Boot”

Unidad 1 - HTTP y J2EE

Bloque 1 - Anatomía de HTTP

Anatomía de una petición HTTP

Cada petición tiene una estructura bien definida

Detrás de cada clic en tu navegador hay una petición HTTP

1 Línea de petición

La primera línea de la petición indica **qué se quiere hacer**, sobre qué recurso y con qué versión de HTTP.

```
GET /productos/1 HTTP/1.1
```

↑ Método ↑ URI ↑ Versión

2 Método

Indica la operación que se quiere realizar sobre el recurso.

GET	Obtener un recurso
POST	Crear un recurso
PUT	Modificar un recurso
DELETE	Eliminar un recurso
PATCH	Modificar parcialmente un recurso

3 URI (Uniform Resource Identifier)

Identifica el recurso al que se quiere acceder. Puede incluir:

- **Path:** /productos/1
- **Query string:** ?categoria=libros
- **Parámetros:** /productos/1

```
/productos?categoria=libros&age=2
```

Path — Query string

4 Versión HTTP

Indica la versión del protocolo que se está utilizando.

HTTP/1.1

Actualmente se utiliza principalmente HTTP/1.1, aunque ya existen versiones posteriores como HTTP/2 y HTTP/3.

5 Cabeceras (Headers)

Proporcionan información adicional sobre la petición (tipo de contenido, navegador, idioma, etc.).

```
Host: www.ejemplo.com
User-Agent: Mozilla/5.0
Accept: text/html,application/xhtml+xml
Accept-Language: es-ES,es;q=0.9
Content-Type: application/json
Content-Length: 123
```

Las cabeceras son opcionales, pero muy habituales.

6 Cuerpo de la petición (Body)

Contiene los datos que se envían al servidor. Es opcional y se utiliza, por ejemplo, en peticiones POST o PUT.

Puede incluir:

- Datos de un formulario
- JSON
- XML
- Archivos (multipart/form-data)
- Texto plano, etc.

7 Ejemplo real

Petición HTTP para crear un producto en formato JSON:

```
POST /productos HTTP/1.1
Host: www.ejemplo.com
User-Agent: Mozilla/5.0
Accept: application/json
Content-Type: application/json
Content-Length: 67

{
  "nombre": "Teclado",
  "precio": 25.90
}
```

Línea de petición — Cabeceras — Cuerpo (body)



Entender la estructura de una petición HTTP nos ayuda a crear mejores aplicaciones web y a resolver problemas.

Conocer cómo se comunican las aplicaciones es el primer paso para construir las tuyas.

Aprende el lenguaje de la Web

Anatomía de una respuesta HTTP

Qué devuelve el servidor y cómo se organiza

El servidor también tiene algo que decirte

1 Status line

La primera línea de la respuesta indica la versión de HTTP, el código de estado y un texto descriptivo (reason phrase).

```
HTTP/1.1 200 OK
```

- `HTTP/1.1` — Versión HTTP
- `200` — Código de estado
- `OK` — Texto descriptivo

2 Código de estado

Indica el resultado de la petición. Son códigos de 3 cifras que se agrupan en categorías.

1xx	Información (poco comunes)
2xx	Éxito (la petición se ha procesado)
3xx	Redirección (otro recurso)
4xx	Error del cliente (petición incorrecta)
5xx	Error del servidor (problema interno)

3 Headers (cabeceras)

Proporcionan información adicional sobre la respuesta (tipo de contenido, tamaño, servidor, caché, etc.).

```
Content-Type: text/html; charset=UTF-8
Content-Length: 1256
Date: Tue, 16 Sep 2025 10:24:00 GMT
Server: Apache/2.4.58 (Unix)
Cache-Control: no-cache
Set-Cookie: sesion=abc123; Path=/; HttpOnly
```

Las cabeceras son pares nombre: valor y cada una va en una línea.

4 Body (cuerpo de la respuesta)

Contiene el recurso solicitado. Puede ser HTML, JSON, una **imagen**, un archivo, etc. Su formato se indica en la cabecera **Content-Type**.

Ejemplos de contenido del body:

- Página web en HTML
- Datos en formato JSON
- Imagen (binario)
- Documento PDF

5 Ejemplo de respuesta HTTP completa

Respuesta del servidor a una petición de una página web:

```
HTTP/1.1 200 OK
```
Status line

```
Date: Tue, 16 Sep 2025 10:24:00 GMT
Server: Apache/2.4.58 (Unix)
Content-Type: text/html; charset=UTF-8
Content-Length: 1256
Cache-Control: no-cache
```
Headers (cabeceras)

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Mi aplicación</title>
</head>
<body>
  <h1>¡Hola, mundo!</h1>
  <p>Esta es una respuesta HTTP de ejemplo.</p>
</body>
</html>
```
Body (cuerpo)



Cada respuesta cuenta una historia

Entender la estructura de una respuesta HTTP te ayuda a depurar problemas, trabajar con APIs y saber qué está pasando en cada petición.

La Web es una conversación

Anatomía de una URL

Cada parte de la URL tiene un significado

Una URL es como una dirección: te dice a dónde ir y qué buscar

1 Esquema

Indica el protocolo que se utiliza para acceder al recurso.

- http (sin cifrado)
- https (con cifrado)
- ftp, mailto, etc.

Ejemplo:

`https://`

2 Host

Indica el nombre de dominio o la dirección IP del servidor donde se encuentra el recurso.



Ejemplo:

`www.ejemplo.com`

3 Puerto

Indica el puerto del servidor. Es opcional y, si no se indica, se utilizan los valores por defecto (80 para HTTP y 443 para HTTPS).

Ejemplo:

`:8080`

4 Path

Indica la ruta del recurso dentro del servidor. Puede incluir varias carpetas.



Ejemplo:

`/productos/1`

5 Query string

Contiene parámetros opcionales para enviar información adicional al servidor. Siempre empieza por ? y los parámetros se separan con &.

Ejemplo:

`?categoria=libros&orden=precio`

6 Fragment

Indica una sección dentro del propio recurso. Se utiliza en páginas HTML para enlazar a un apartado concreto. Empieza por # y no se envía al servidor.

Ejemplo:

`#descripcion`

7 Estructura completa

Una URL puede incluir todas las partes o solo algunas de ellas.

`https://` `www.ejemplo.com` `:8080` `/productos/1` `?categoria=libros&orden=precio` `#descripcion`

Esquema	Host	Puerto (opcional)	Path	Query string (opcional)	Fragment (opcional)

8 Más ejemplos

`https://www.google.com`	URL simple, sin puerto, ni parámetros, ni fragmento.
`http://localhost:8080/app`	Con puerto y path.
`https://www.ejemplo.com/buscar?q=java&pagina=2`	Con query string (parámetros).
`https://www.ejemplo.com/docs/introduccion.html#uso`	Con fragmento.
`https://api.ejemplo.com:8443/v1/productos/15?formato=json#respuesta`	URL completa con todas las partes.



Una URL es mucho más que una dirección: es la clave para encontrar recursos en la Web.

Conocer la estructura de una URL te ayuda a entender cómo funcionan las aplicaciones web y a construir tus propias URL.

Pequeñas piezas que conectan un gran mundo

Explora, aprende y construye tu Web

Los recursos en HTTP

La Web está formada por recursos que se identifican con una URL

Cada recurso tiene su propia URL

1 ¿Qué es un recurso?

Un recurso es cualquier elemento al que se puede acceder a través de una **URL**. Puede ser un documento, una imagen, un dato, un producto, un usuario, etc.



En HTTP, todo es un recurso.

2 Recursos frente a páginas

Una página web es una representación visual de uno o varios recursos (normalmente en formato HTML).
Un recurso puede tener varias **representaciones:** HTML para un navegador o JSON para una API.

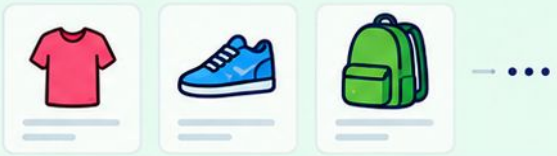


El mismo recurso, diferentes representaciones.

3 Colecciones

Una colección es un conjunto de recursos del mismo tipo. Se suele acceder mediante una URL en plural.

`/productos`



`GET /productos` → devuelve la lista de productos (normalmente en formato JSON).

4 Recursos individuales

Cada recurso de la colección tiene su propia URL única, normalmente añadiendo un identificador.

`/productos/15`



Producto #15
Nombre: Camiseta
Precio: 19,90 €
Stock: 25 unidades

`GET /productos/15` → devuelve los datos producto con identificador 15.

5 Relación colección - recurso

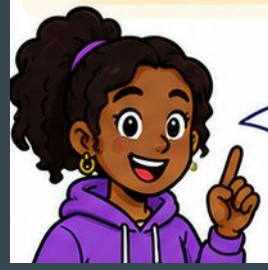
Una colección contiene varios recursos individuales. Desde la colección podemos obtener, crear, modificar o eliminar recursos.



6 Más ejemplos

Este mismo patrón se aplica a todo tipo de recursos:

`/usuarios`	(colección)
`/usuarios/8`	(recurso individual)
`/pedidos`	(colección)
`/pedidos/27`	(recurso individual)
`/categorias`	(colección)
`/categorias/4`	(recurso individual)



Las URLs nos permiten identificar y acceder a recursos de forma sencilla. ¡Es como tener una dirección única para cada cosa en la Web!

Entender los recursos en HTTP es clave para diseñar y consumir APIs RESTful.

Recursos bien diseñados, aplicaciones más simples y potentes.

La Web está llena de recursos. ¡Explora sus posibilidades!